

EX,NO.06	WEBSITE DEVELOPMENT AND PORTFOLIO TIC TAC TOE GAME
DATE:	

AIM:

To design, develop, and deploy a browser-based Tic Tac Toe game as a website using HTML, CSS, and JavaScript, and to host the project on GitHub with a working live demo.

OVERVIEW:

Tic-Tac-Toe Website is a simple, browser-based two-player game. Two players take turns placing X and O on a 3×3 grid. The game checks rows, columns, and diagonals to determine a winner, displays the game status, detects a draw, and provides a Restart button for a new game.

STEPS / PROCEDURE

1. Created a new project folder containing three files: index.html (structure), style.css (styling), and script.js (game logic).
2. Built the HTML structure with a title, game status, a 3×3 board containing nine cells, and a Restart button.
3. Styled the page in CSS with a clean colourful design, centered layout, rounded cells, and clear X/O text.
4. In script.js, created a board array containing nine empty positions and a variable to track the current player.
5. Attached a click handler to each board cell so the current player can place X or O.
6. Checked all possible winning combinations: three rows, three columns, and two diagonals.
7. If a winning combination is completed, displayed the winner and stopped further moves.
8. If all nine cells are filled without a winner, displayed a Draw message.
9. Updated the game status after every move to show whose turn it is.
10. Implemented the Restart button to clear the board and reset the current player to X.
11. Tested horizontal, vertical, diagonal, and draw conditions.
12. Initialized a Git repository, committed the project files, and pushed the code to GitHub under the repository name Tic-Tac-Toe-Website.
13. Wrote a README.md documenting the project overview, live demo, how to play, features, and technologies used.
14. Deployed the website and verified that the hosted version works correctly.



CODE:

Core files from the project(index.html,script.js,style.css),showing the page structure,game logic,and styling:

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Tic Tac Toe</title>

  <style>

    * {

      box-sizing: border-box;

    }

    body {

      margin: 0;

      font-family: Arial, sans-serif;

      background: white;

      text-align: center;

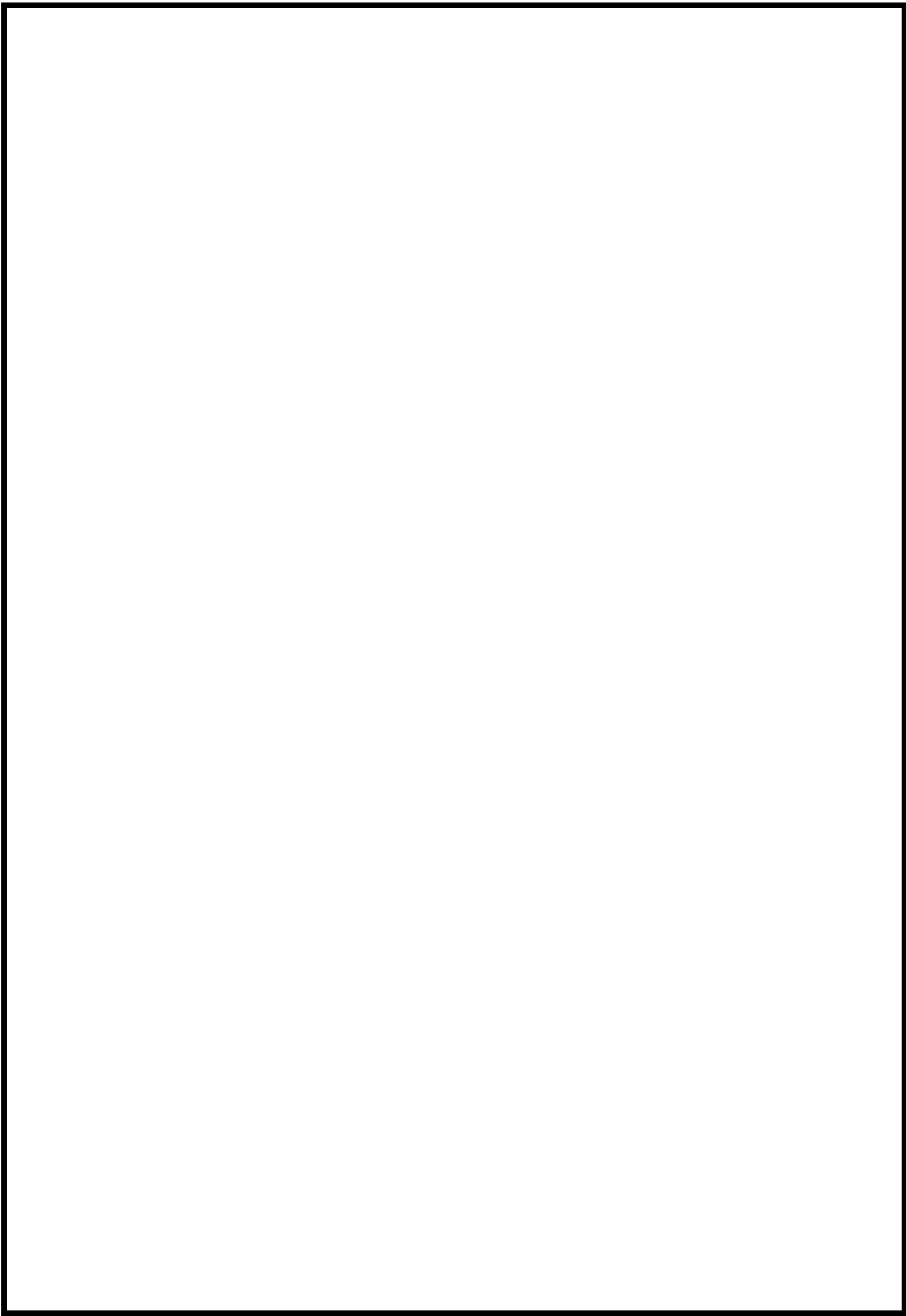
    }

    .topbar {

      height: 80px;

      background: #3f51b5;

      color: white;
```



```
display: flex;
er;
justify-content: space-between;

padding: 0 25px;
}
```

```
.title {
  font-size: 30px;
  font-weight: bold;
}
```

```
.menu {
  font-size: 30px;
}
```

```
.game {
  padding: 40px 15px;
}
```

```
#status {
  font-size: 24px;
  font-weight: bold;
  margin-bottom: 30px;
}
```

```
.board {
  width: 90vw;
  max-width: 360px;
```

margin: auto;

```
display: grid;
  grid-template-columns: repeat(3, 1fr);
```

```
  gap: 10px;
}
```

```
.cell {
  height: 110px;

  background: #f5f5f5;

  border: 2px solid #3f51b5;

  border-radius: 8px;

  font-size: 55px;
  font-weight: bold;

  color: #222;

  cursor: pointer;
}
```

```
.cell:active {
  background: #e8eaf6;
}
```

```
#reset {
  margin-top: 30px;
```

padding: 13px 30px;

```
background: #3f51b5;
  color: white;

border: none;
border-radius: 6px;

font-size: 18px;

cursor: pointer;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<!-- HTML PART -->
```

```
<div class="topbar">
```

```
<div class="title">
```

```
  Tic Tac Toe
```

```
</div>
```

```
<div class="menu">
```

```
  ≡
```

```
</div>
```

</div>

```
<div class="game">
```

```
  <div id="status">
```

```
    Player X's Turn
```

```
  </div>
```

```
  <div class="board">
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
    <button class="cell"></button>
```

```
  </div>
```

```
  <button id="reset">
```

```
    New Game
```

```
  </button>
```

```
</div>
```



```
<!-- JAVASCRIPT PART -->
```

```
<script>
```

```
// Get all 9 buttons
```

```
const cells = document.querySelectorAll(".cell");
```

```
// Get status message
```

```
const status = document.getElementById("status");
```

```
// Get New Game button
```

```
const resetButton = document.getElementById("reset");
```

```
// Store the 9 cells
```

```
let board = [
```

```
  "", "", "",
```

```
  "", "", "",
```

```
  "", "", ""
```

```
];
```

```
// First player
```

```
let currentPlayer = "X";
```

```
// Game is active
```

```
let gameActive = true;
```

// All possible winning combinations

```
const winningCombinations = [  
  
    [0, 1, 2],  
    [3, 4, 5],  
    [6, 7, 8],  
  
    [0, 3, 6],  
    [1, 4, 7],  
    [2, 5, 8],  
  
    [0, 4, 8],  
    [2, 4, 6]  
  
];  
  
// Add click action to every cell  
cells.forEach(function(cell, index) {  
  
    cell.addEventListener("click", function() {  
  
        // Do nothing if game is over  
        if (gameActive === false) {  
            return;  
        }  
  
        // Do nothing if cell already contains X or O  
        if (board[index] !== "") {
```

```
return;
```

```
}
```

```
// Put X or O in the cell
```

```
board[index] = currentPlayer;
```

```
cell.textContent = currentPlayer;
```

```
// Check whether someone won
```

```
if (hasWinner()) {
```

```
    status.textContent =
```

```
        "Player " + currentPlayer + " Wins!";
```

```
    gameActive = false;
```

```
    return;
```

```
}
```

```
// Check whether the game is a draw
```

```
if (!board.includes("")) {
```

```
    status.textContent =
```

```
        "It's a Draw!";
```

```
    gameActive = false;
```

```
    return;
```



```
}

// Change player
if (currentPlayer === "X") {

    currentPlayer = "O";

} else {

    currentPlayer = "X";

}


// Display next player's turn
status.textContent =
    "Player " + currentPlayer + "'s Turn";

});

});


// Function to check winner
function hasWinner() {

    // Check all 8 winning combinations
    for (let combination of winningCombinations) {
        let a = combination[0];
```



```
let b = combination[1];
```

```
let c = combination[2];
```

```
if (  
    board[a] !== "" &&  
    board[a] === board[b] &&  
    board[a] === board[c]  
) {
```

```
    return true;
```

```
}
```

```
}
```

```
return false;
```

```
}
```

```
// New Game button
```

```
resetButton.addEventListener("click", function() {
```

```
// Clear all 9 cells
```

```
board = [  
    "", "", "",
```



```
"" , "" , ""
```

```
];
```

```
// Start with X
```

```
currentPlayer = "X";
```

```
// Activate game
```

```
gameActive = true;
```

```
// Clear the buttons
```

```
cells.forEach(function(cell) {
```

```
    cell.textContent = "";
```

```
});
```

```
// Reset status
```

```
status.textContent =
```

```
    "Player X's Turn";
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```



WEBSITE FLOW WITH SCREENSHOTS:



Fig:6.1- website in its initial score state :scoreboard at 0/0 , two move buttons, and “choose your move !” prompt.



Tic Tac Toe



Player O Wins!

X	X	O
X	O	
O		

New Game



S.NO	CATEGORIES	MARKS ALLOTED	MARKS OBTAINED
1	CONCEPTUAL KNOWLEDGE/SKILL	20	
2	PRESENTATION/ IMPLEMENTATION	20	
3	OUTPUT/TESTING	15	
4	VIVA	10	
5	RECORD	10	
6	TOTAL	75	

RESULT:

A browser-based Rock Paper Scissors website was successfully designed, developed, and deployed using HTML, CSS, and JavaScript. The website correctly determines the win/lose/tie outcome for every round, tracks and persists the score using local Storage, and was published on GitHub with a working live demo link.

